LEZIONE 2

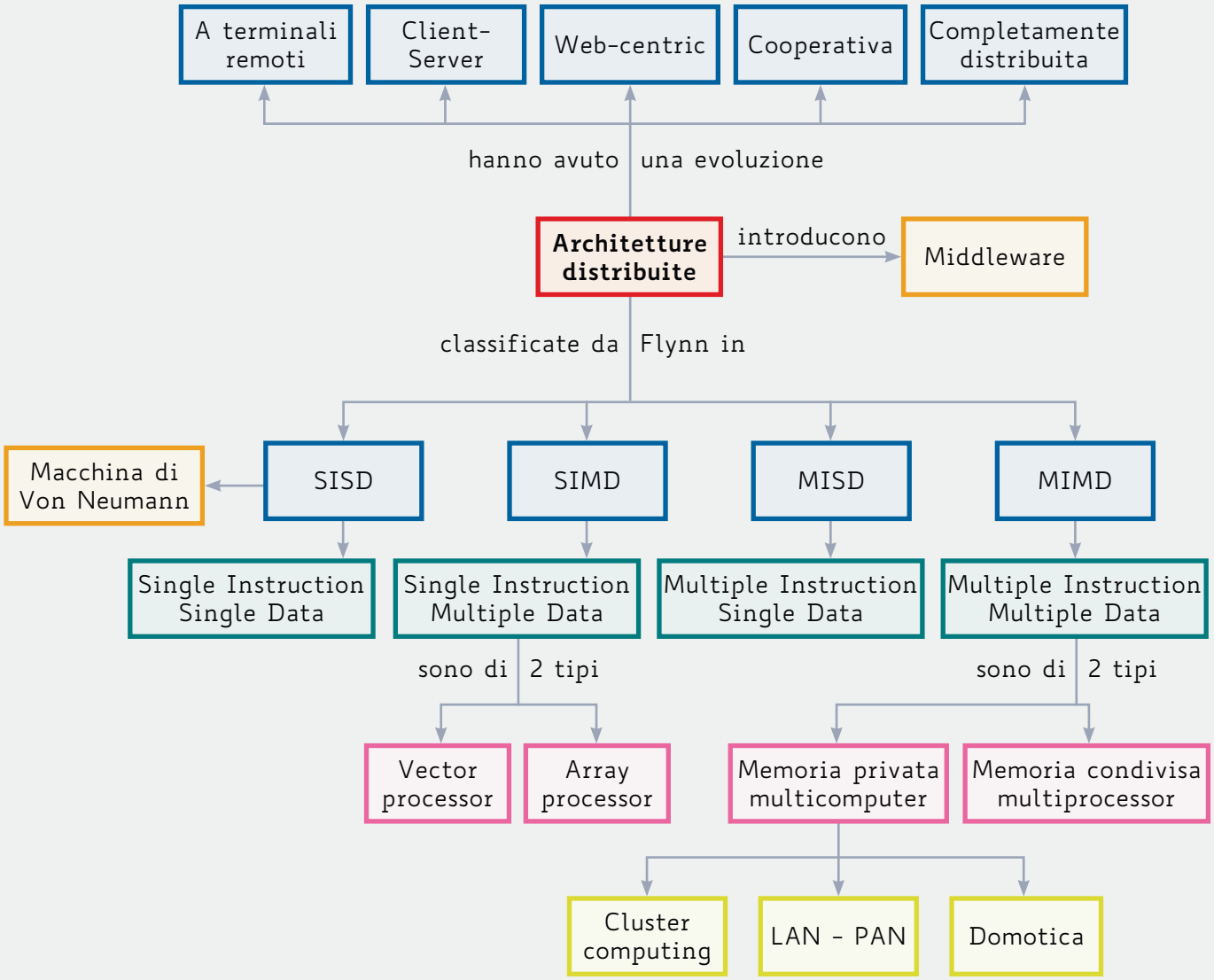
2 Evoluzione dei sistemi distribuiti e dei modelli architetturali

IN QUESTA LEZIONE IMPAREREMO...

- la classificazione delle architetture distribuite hardware e software
- il concetto di middleware

MAPPA CONCETTUALE

didattica inclusiva



Premessa

Un sistema distribuito è ottenuto dalla connessione di più personal computer che per molti anni hanno avuto una crescita tecnologica legata alla legge di Moore ma che nel prossimo futuro sarà disattesa in quanto non sarà più possibile raddoppiare la potenza degli elaboratori ogni due anni.

> Il problema maggiore è quello legato alla miniaturizzazione dei microprocessori e alla crescita delle frequenze di lavoro!

Attualmente i microprocessori più potenti hanno circuiti larghi 14 nanometri (un miliardesimo di metro) e si sta lavorando per farne di ancora più piccoli, ma ci sono limiti fisici con i quali si dovrà fare i conti quando si arriverà nell'ordine dei 2-3 nanometri dato che a quelle dimensioni i componenti sarebbero larghi quanto una decina di atomi!
Le leggi della fisica pongono dei limiti anche a partire dal fatto che non è possibile superare la velocità della luce e quindi, dato che questa nel rame è di circa 200.000 km/s, si ha:

$$v = \frac{200000 \cdot 10^3}{10^9} = 0,2 \text{ [m/ns]}$$

Questo significa che nel tempo di 1 ns vengono percorsi 20 cm di conduttore.
L'unica soluzione è quelle di andare quindi verso architetture di elaborazione di diverso tipo, sia dal punto di vista costruttivo (hardware) cha del punto di vista logico (software).

> Se vogliamo avere frequenze di lavoro dell'ordine dei Ghz non possiamo superare la distanza di 20 cm in trasmissione altrimenti verrebbe generato un ritardo e se si restringono i circuiti avremmo problemi di instabilità e di surriscaldamento in quanto il calore prodotto viene difficilmente dissipato in una architettura così densa.

Architetture distribuite hardware: dalle SISD al cluster di PC

Dal punto di vista hardware si è passati alla realizzazione di macchine e sistemi dotati di più di una CPU in modo da avere più potenza di calcolo senza "esasperare" i limiti di velocità di ogni singola CPU: stiamo parlando di macchine parallele o macchine ad architettura parallela.
Esistono diverse possibilità per classificare le architetture hardware a seconda dei fattori che si prendono come riferimento. Noi ricordiamo quella di Flynn (1972) che si basa sui due flussi di informazioni normalmente presenti nei calcolatori:

- flusso delle istruzioni;
- flusso dei dati.

A seconda di come si "combinano" flusso di dati e di istruzioni abbiamo quattro possibili situazioni:

	DATI SINGOLI	DATI MULTIPLI
Istruzioni singole	SISD	SIMD
Istruzioni multiple	MISD	MIMD

- macchine SISD (Single Instruction Single Data): flusso di istruzioni unico e flusso di dati unico;
- macchine SIMD (Single Instruction Multiple Data): flusso di istruzioni unico e flusso di dati multiplo;
- macchine MISD (Multiple Instruction Single Data): flusso istruzioni multiplo e flusso dati unico;
- macchine MIMD (Multiple Instruction Multiple Data): flusso istruzioni multiplo e flusso dati multiplo.

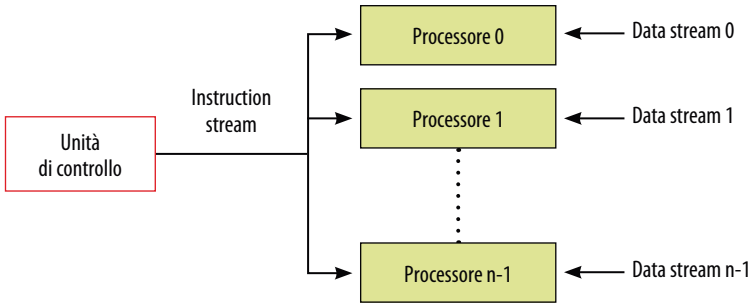
SISD

Un elaboratore come la macchina di Von Neumann che ha un solo flusso dati e un solo flusso istruzioni è una macchina Single Instruction stream – Single Data stream (SISD): tutte le macchine che hanno una singola CPU come i personal computer, le workstation e i mainframe appartengono a questa categoria.

Nelle macchine a singola CPU il flusso di istruzioni è unico e quindi viene eseguito un solo programma alla volta: dopo l'esecuzione della prima istruzione si passa alla seconda e così via fino al termine del programma, le istruzioni sono quindi eseguite in modalità sequenziale.

SIMD

Nelle macchine SIMD l'elaborazione avviene su più flussi dati in contemporanea ma con un singolo flusso di istruzioni: generalmente sono presenti più processori (macchine a vector processor o ad array processor) che eseguono la stessa istruzione su flussi di dati diversi (si sfrutta il parallelismo a livello di dati).



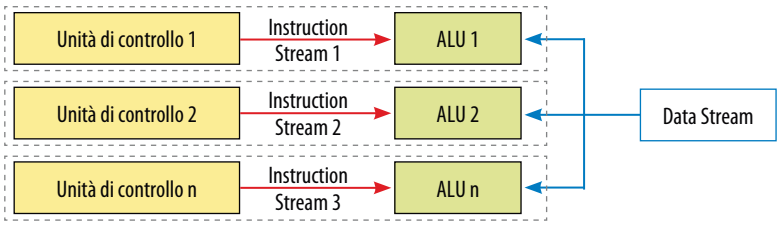
Un elaboratore SIMD è particolarmente adatto per realizzare calcoli vettoriali e matriciali soprattutto in ambiente grafico.

MISD

A questa categoria appartengono gli elaboratori che eseguono più istruzioni sullo stesso flusso dati, cioè Multiple Instruction stream-Single Data stream (MISD): nel modello di calcolo MISD esistono quindi più processori, ognuno con una propria memoria (registri), la quale a sua volta avrà un proprio flusso di istruzioni che verranno eseguite sullo stesso flusso di dati.

Con questa tipologia di architettura a oggi non sono ancora state costruite macchine da commercializzare, anche se il modello ricorda la modalità di esecuzione delle istruzioni in pipeline.

Avere a disposizione una macchina con questa caratteristiche potrebbe essere utile nell'ambito della crittografia, dove vengono richieste elaborazioni simili da eseguirsi sugli stessi dati in parallelo.

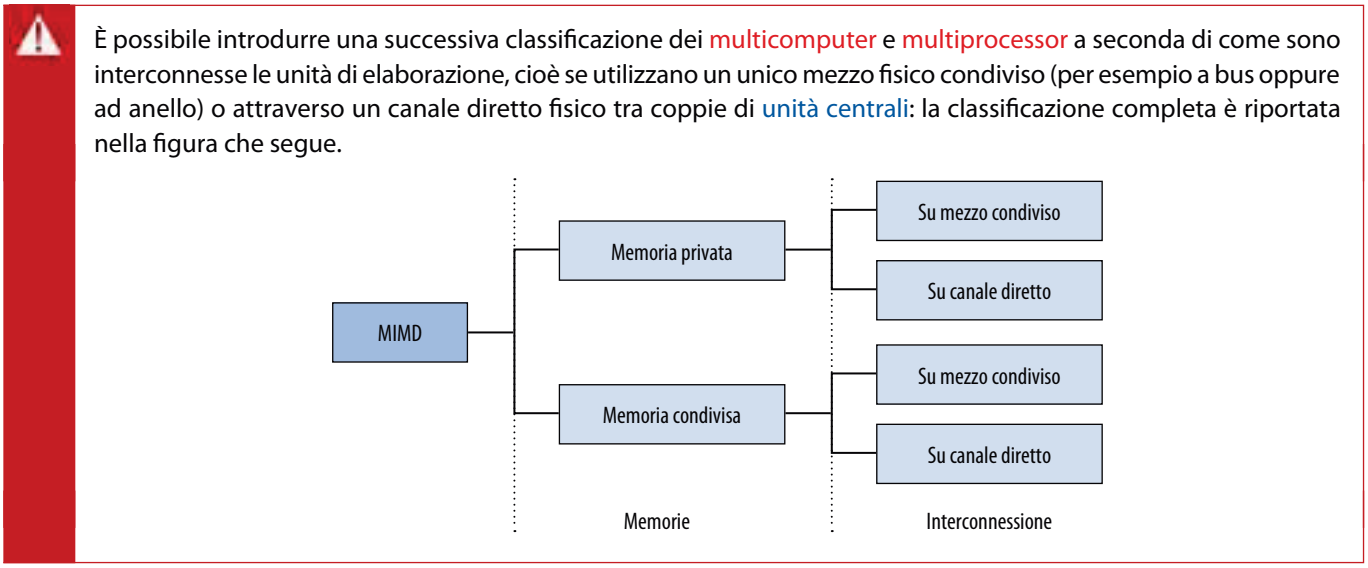


MIMD

L'architettura Multiple Instruction stream-Multiple Data stream (MIMD) comprende tutte le tipologie di elaboratori composti da più unità centrali di elaborazione indipendenti che possono lavorare su stream di dati anch'essi indipendenti. Viene effettuata una ulteriore classificazione delle macchine MIMD in riferimento a come è suddivisa la memoria fisica, e cioè possiamo avere:

- macchine MIMD a memoria fisica condivisa;
- macchine MIMD a memoria privata.

Le prime sono anche conosciute con il nome di multiprocessor, mentre le seconde con quello di multicomputer.

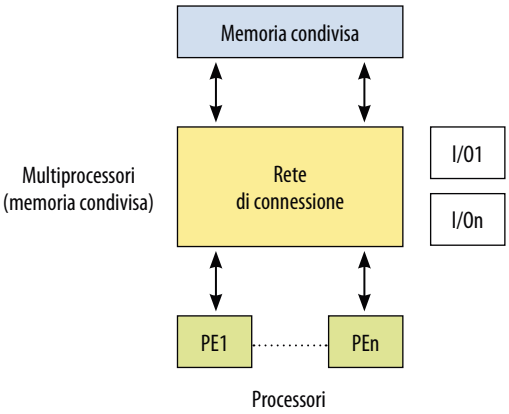


È possibile introdurre una successiva classificazione dei multicomputer e multiprocessor a seconda di come sono interconnesse le unità di elaborazione, cioè se utilizzano un unico mezzo fisico condiviso (per esempio a bus oppure ad anello) o attraverso un canale diretto fisico tra coppie di unità centrali: la classificazione completa è riportata nella figura che segue.

In queste macchine sia il flusso delle istruzioni che dei dati è multiplo: ogni processore legge le proprie istruzioni e opera sui propri dati e quindi sono presenti contemporaneamente più processi in esecuzione. Il parallelismo è fisico ed è molto flessibile dato che ogni singola CPU evolve in modo autonomo.

MIMD: multiprocessori

I sistemi a multiprocessori sono architetture MIMD a memoria condivisa (shared memory): la memoria è in comune, e quindi esiste un unico spazio di indirizzamento condiviso tra tutti i processori.

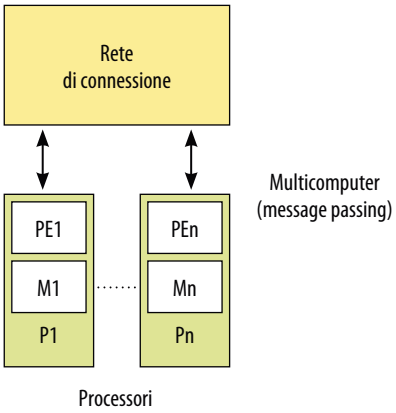


Dato che la comunicazione tra processi avviene mediante variabili condivise è necessario implementare gli opportuni meccanismi di sincronizzazione per regolare gli accesi alla memoria in modo da coordinare i diversi processi per gestire la competizione alle risorse comuni.

Non necessariamente i processori devono avere una unica memoria in comune (centralizzata): possono avere ciascuno una propria memoria e condividerne parte con gli altri processori, in modo da realizzare una memoria condivisa distribuita.

MIMD: multicomputer

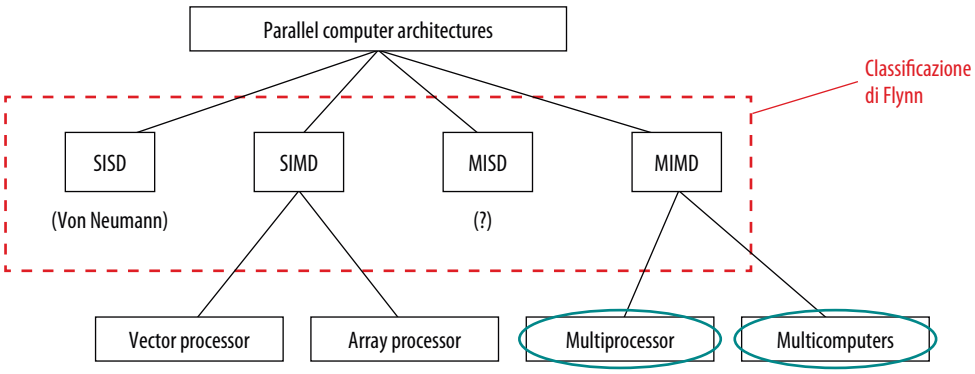
Nei multicomputer non è presente una memoria condivisa a livello di architettura e quindi la comunicazione avviene mediante lo scambio di messaggi (message passing) esplicito, effettuato mediante apposite procedure (send e receive). Ogni computer possiede una propria area di memoria privata, non indirizzabile da parte dei processori remoti.



ESEMPIO

Le LAN di un personal computer sono da considerarsi sistemi MIMD dato che hanno una memoria privata e sono tra loro interconnessi con mezzo condiviso.

Possiamo ora estendere la classificazione di Flynn inserendo anche le sottoclassificazioni:



Cluster computing

Con cluster computing si intende un sistema distribuito costituito da un insieme di nodi ad alte prestazioni interconnessi tramite una rete locale ad alta velocità: devono essere omogenei, cioè i singoli nodi hanno lo stesso sistema operativo, hardware molto simile, e sono connessi attraverso la stessa rete.
Teoricamente un cluster di PC ha una potenza di calcolo pari alla somma di quelle dei singoli computer che lo costituiscono, e differisce da una rete di PC principalmente per:

- la potenza di elaborazione ad alte prestazioni (HPC o High Performance Computing);
- la velocità del trasferimento dati (oltre 1 Gbit/s);
- la centralizzazione fisica delle macchine, tutti i PC sono montati sullo stesso rack;
- la presenza di una applicazione di management, residente su un singolo PC, che permette di lanciare processi su altri PC, monitorare il loro comportamento ecc.

AREA DIGITALE
Creare cluster HPC con ParallelKnoppix

Abbiamo due tipiche possibili architetture:

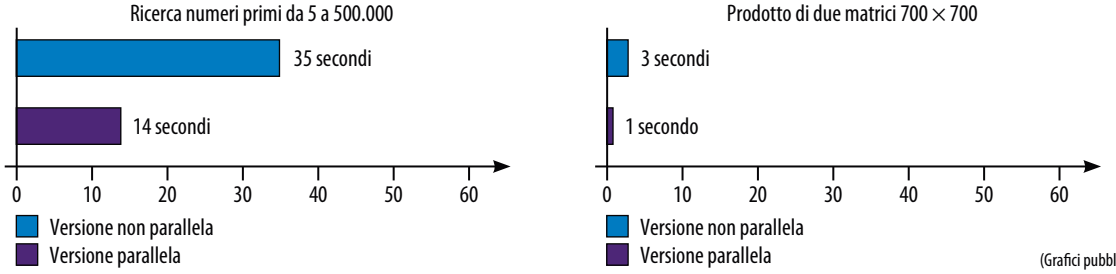
1. organizzazione gerarchica con singolo nodo principale: per esempio Beo/wulf, dove spesso sono usate librerie di message passing per il calcolo parallelo (MPI);
2. organizzazione Single System Image: per esempio MOSIX, che effettua un bilanciamento automatico del carico effettuando una eventuale migrazione di processi.

In base alla definizione e considerando le unità centrali come entità, un sistema cluster di PC corrisponde all'insieme delle macchine MIMD a memoria privata.

L'enorme vantaggio dei cluster di PC è quello di affrontare calcoli particolarmente onerosi che sarebbero molto lunghi o impossibili con un solo computer.

ESEMPIO

Vediamo per esempio il confronto di prestazioni ottenuto con il calcolo parallelo effettuato su un cluster di 4 PC in rispetto all'elaborazione sequenziale in due classiche situazioni di calcolo matematico: *ricerca di numeri primi e il prodotto di matrici*.
Come è possibile osservare dai grafici riportati di seguito, nella ricerca dei numeri primi si ha una riduzione del tempo di elaborazione da 35 secondi a 14 secondi, mentre per il prodotto di due matrici di ordine 700 il tempo di computazione scende da 3 secondi a 1 secondo!



(Grafici pubblicati su Rottanuova)

Grid computing

Con grid computing si intende un sistema distribuito di calcolo altamente decentralizzato, composto da un gran numero di nodi disposti a griglia (grid) e caratterizzati da un grado elevato di eterogeneità sia per hardware, per il software, la tecnologia di rete, le politiche di sicurezza ecc.
Il termine "griglia" è stato coniato intorno alla metà degli anni Novanta: il vero problema alla base del concetto di griglia è la condivisione coordinata di risorse all'interno di una dinamica e multi-istituzionale organizzazione virtuale (Virtual Organization, brevemente indicata con VO).

> La condivisione non è limitata solo allo scambio dei file, ma si estende all'accesso diretto a computer, al software, in generale a tutto l'hardware necessario alla risoluzione di un problema scientifico, ingegneristico o industriale.

Si è passati da un'architettura a 4 livelli di specificità alla sua evoluzione service-oriented che ha portato alla definizione della Open Grid Service Architecture (OGSA) dove risorse computazionali, di storage, reti, programmi, database ecc., sono tutti dei servizi (grid services).

Sistemi distribuiti pervasivi

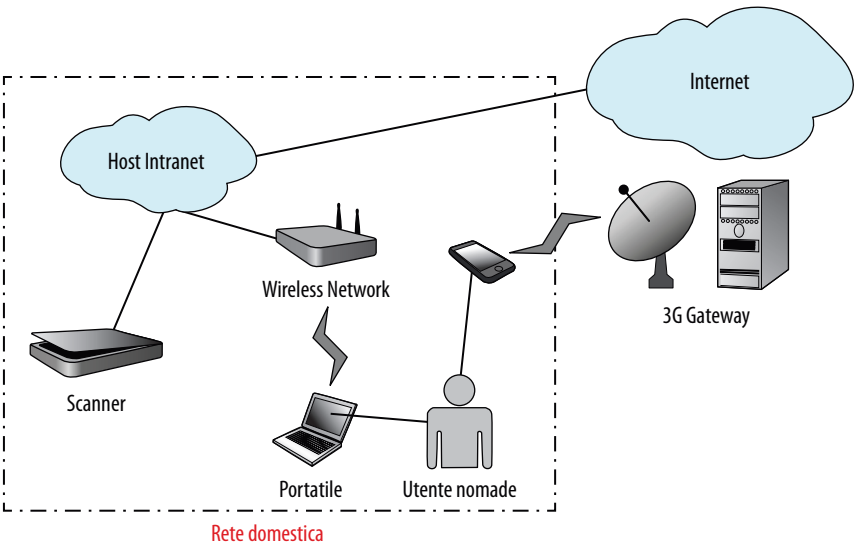
Questa è una nuova generazione di SD i cui nodi sono piccoli, mobili, con connessioni di rete wireless e spesso facenti parte di un sistema più grande:
- sistemi domestici, sistemi elettronici per l'assistenza sanitaria;
- reti di sensori.

Alcuni requisiti per sistemi pervasivi:
- *cambi di contesto*: il sistema è parte di un ambiente che può cambiare in ogni momento;
- *composizione ad hoc*: ogni nodo può essere usato in modi molto diversi da utenti differenti;
- richiesta la facilità di configurazione;
- *condivisione come default*: i nodi vanno e vengono, fornendo informazioni e servizi da condividere.

Pervasività e trasparenza della distribuzione non sono facilmente coordinabili: è preferibile esporre la distribuzione piuttosto che cercare di nasconderla.

Reti domestiche e domotica

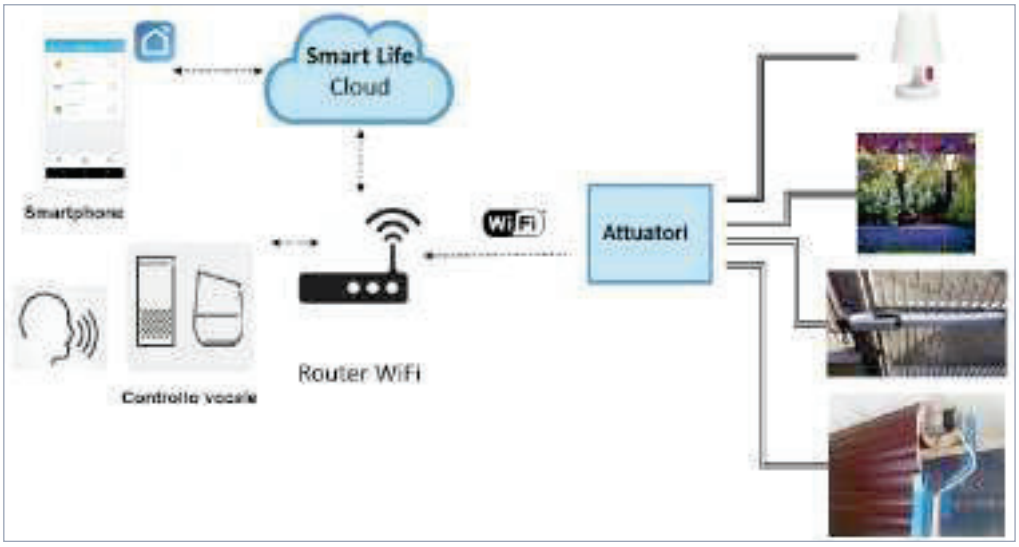
Le reti domestiche sono caratterizzate dall'assenza di un amministratore di sistema e generalmente sono sistemi completamente auto-configuranti e autogestiti in quanto generalmente gli utenti non hanno conoscenze specifiche di connettività.
La soluzione più semplice e quella di una home box centralizzata alla quale si connettono wireless tutte le periferiche ed eventuali dispositivi "ospiti".



L'espansione d'ambito applicativo che sta investendo l'IT con l'interesse crescente verso l'integrazione del mondo fisico con quello digitale ha dato un forte impulso innovativo alla domotica favorendone una rapida evoluzione tecnologica. Anche le esigenze dei consumatori hanno fatto segnare una aumentata richiesta di intelligenza della casa e l'attesa di ritrovare negli impianti domestici gli stessi pattern di interazione e semplicità tipici degli smartphone.
La domotica è un neologismo nato da "*domus*" (casa) e "automatica" e si occupa delle tecnologie capaci di migliorare la qualità di vita all'interno della casa.
Un modello che meglio descrive l'impianto domotico contemporaneo è quello introdotto da D.J.Cook ovvero l'Ambient Intelligence proposto con la definizione che segue:

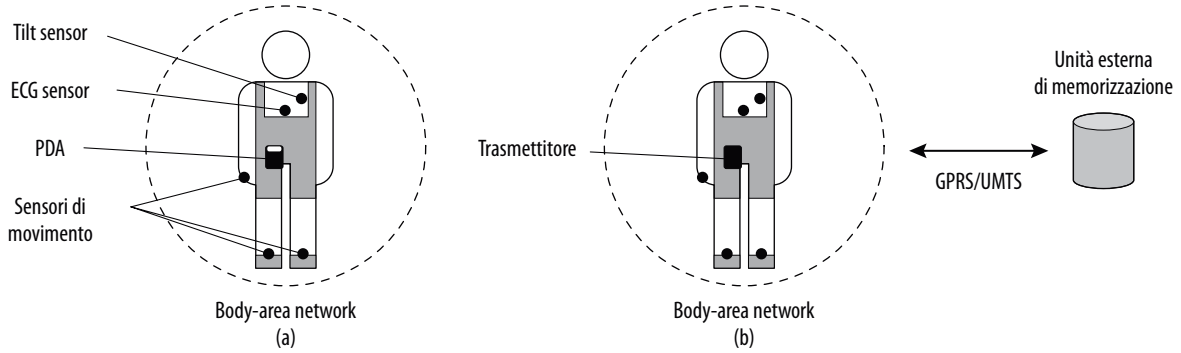
"an Ambient Intelligence system is a digital environment that proactively, but sensibly, supports people in their lives"

che possiamo tradurre liberamente come: "un sistema che proattivamente, ma in modo ragionato, supporta le persone nella loro vita".
La domotica consente di ottenere un notevole incremento delle prestazioni e delle possibilità offerte dai diversi impianti presenti nell'abitazione, ottimizzando i consumi e permettendo l'integrazione di diverse funzioni quali controllo, comfort, sicurezza, risparmio energetico e comunicazione.
Generalmente una rete domotica è senza fili e comunica con tutti i dispositivi compatibili presenti fra le mura domestiche: l'utilizzatore può quindi automatizzare tutte quelle operazioni che svolge quotidianamente come, per esempio, accendere le luci, alzare le tapparelle, regolare la temperatura della propria abitazione o gestire l'allarme domestico sia da smartphone sia con le nuove tecnologie di controllo vocale (Alexa, Domoki, Apple Home, Google Home ecc.).

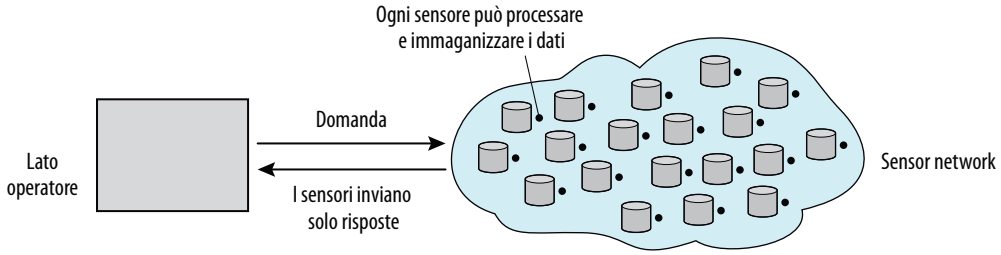


Wearable computing

Oggi i sistemi wearable computing sono essenzialmente utilizzati per l'assistenza sanitaria ed effettuano il monitoraggio di parametri biologici o memorizzando i dati in un computer palmare (PDA *Personal Digital Assistant*) come nell'esempio riportato nel disegno a) oppure trasmettendo i dati a un sistema di archiviazione remoto, come nel caso b).



I principali problemi che vanno affrontati in questi sistemi, oltre alla memorizzazione dei dati è la prevenzione contro la perdita di dati cruciali, la gestione della sicurezza e le modalità di generazione e propagazione di eventuali allarmi.
Le reti di sensori sono composte da molteplici semplici sensori (da 10 a 1000) e ogni sensore corrisponde a un nodo della rete: generalmente i dati raccolti vengono elaborati e memorizzati in una base di dati centralizzata ma in alcuni casi i dati vengono proprio memorizzati in essi stessi, realizzando di fatto anche una base di dati distribuita.

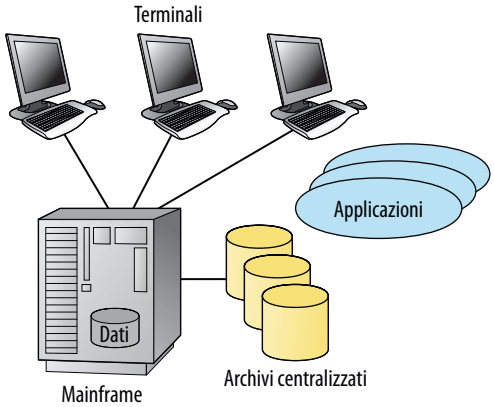


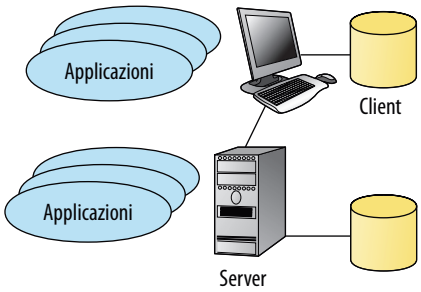
Architetture distribuite software: dai terminali remoti ai sistemi completamente distribuiti

Come per l'hardware, anche per il software abbiamo avuto una evoluzione nelle architetture distribuite che spesso hanno anticipato i cambiamenti delle architetture hardware e dei loro sistemi operativi. Ricordiamo brevemente le tappe fondamentali.

Architettura a terminali remoti

La prima tipologia di elaboratori ha questa architettura dove tutte le operazioni vengono effettuate da un'unica entità centrale alla quale sono collegati terminali privi di capacità di elaborazione che si limitano a visualizzare e inviare e ricevere le informazioni alla entità centrale.
In questi sistemi si hanno terminali omogenei e quindi il software di gestione svolge dei compiti relativamente semplici, replicando per ciascuno di essi una area di memoria riservata e facendo evolvere singolarmente i singoli task avviati dalle diverse unità remote.





Architettura client-server

A differenza dell'architettura precedente i client non sono "macchine stupide" ma hanno una loro capacità di elaborazione: questi richiedono un servizio a un server "di competenza", che è in grado di esaudirne la richiesta, la elabora e invia la risposta al client. Come vedremo in seguito, possono essere presenti più server che offrono più servizi contemporaneamente così come più client possono richiedere lo stesso servizio o servizi differenti sia a server diversi che allo stesso server.

Inoltre un server può essere contemporaneamente anche client: quindi può essere server per uno (o più) servizi e client per altri richiedendo i servizi che necessita ad altri server.
Client e server possono essere tecnologicamente diversi, sia come hardware che come sistema operativo: questa architettura è quella che meglio si presta a far comunicare e cooperare entità non omogenee.

Architettura WEB-centric

L'evoluzione che negli ultimi anni ha caratterizzato i sistemi distribuiti riguarda il Web: la diffusione capillare delle piattaforme mobili ha, di fatto, provocato uno spostamento delle applicazioni sul server facendo "in qualche modo" regredire gli host quasi "a terminali stupidi del secolo passato", utilizzati solo per l'interfaccia grafica verso l'utente.
Molte applicazioni gestionali sono state "convertite al Web" rendendo il Web server come centro del sistema distribuito (Web-centric) al quale i client vi accedono per ottenere dei servizi: tutta la computazione avviene sui server che restituisce ai client il risultato.
I componenti fondamentali di un'applicazione Web sono analoghi per certi versi a quelli di una tradizionale applicazione client-server: infatti il Web è un sistema distribuito composto da client e server che accedono a documenti collegati. I server gestiscono insiemi di documenti, mentre i client forniscono agli utenti una interfaccia facile da usare per accedere e presentare questi documenti.
Le architetture possono essere classificate in:
- architetture Web tradizionali;
- architetture Web multilivello.

Architettura cooperativa

L'evoluzione dell'architettura client-server è l'architettura cooperativa che si basa su entità autonome che esportano e richiedono servizi secondo il modello di sviluppo a componenti per la programmazione lato server proprio della programmazione a oggetti.
Di fatto è stato preso uno dei principi chiave della programmazione orientata agli oggetti, l'incapsulamento, applicandolo all'abbattimento delle differenze tra i prodotti usati nell'infrastruttura hardware, software, di programmazione e di rete di un sistema distribuito.
Per rendere efficace l'utilizzo di questi meccanismi è necessario introdurre una standardizzazione sulle modalità con le quali i servizi vengono offerti e come i client possano richiederli: OdP (Open Distributed Processes) e CORBA (Common Object Request Broker Architecture) sono due esempi di standardizzazione.

Architettura completamente distribuita

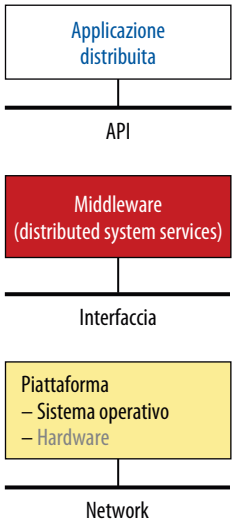
In opposizione alla architettura Web-centric troviamo l'architettura completamente distribuita, tipica di sistemi dove è necessaria la cooperazione di gruppi di entità paritetiche, come nei sistemi groupware, che dialogano tra loro offrendo ciascuno i propri servizi e richiedendo servizi che spesso risultano essere duplicati per garantire l'immunità ai guasti (vengono introdotte ridondanze anche nei dati).
Le tecnologie più importanti sono:
- OMG (Object Management Group): è un consorzio creato nel 1989 da varie aziende tra cui Microsoft, HP, NCR, SUN, allo scopo di creare sistemi di gestione di architetture distribuite;
- RMI (Remote Method Invocation): è una tecnologia che consente a processi Java distribuiti di comunicare attraverso una rete; è specifica appunto del mondo Java;
- DCOM (Distributed Component Object Model): è una tecnologia informatica introdotta nel 1996 da Microsoft ed è basato sull'estensione in rete della precedente tecnologia COM.

Architettura a livelli

Per alleggerire il carico elaborativo dei serventi sono state introdotte le applicazioni multilivello, nelle quali avviene la separazione delle funzionalità logiche del software in più livelli.
Si introducono gli *strumenti di* middleware, cioè uno strato software "in mezzo" che si colloca sopra al sistema operativo ma sotto i programmi applicativi, e sono l'evoluzione dei sistemi operativi distribuiti.

Con middleware si intende una classe di tecnologie software sviluppate per aiutare gli sviluppatori nella gestione della complessità e della eterogeneità presenti nei sistemi distribuiti (D.E. Bakken). Il middleware ha lo scopo di realizzare la comunicazione e le interazioni tra i diversi componenti software di un sistema distribuito.

Questo software si interpone quindi negli elaboratori locali tra le applicazioni e il sistema operativo locale creando un'architettura a tre livelli, può quindi essere diverso da host a host, e sarà descritta nella prossima Lezione.
Lo scopo principale di middleware è di permettere e garantire l'interoperabilità delle applicazioni sui diversi sistemi operativi.
Inoltre permette la connettività tra servizi che devono interagire e collaborare su piattaforme distribuite sulla base di meccanismi di programmazione e API relativamente semplici.
Di fatto la presenza di questo strato rende facilmente programmabili i sistemi distribuiti offrendo una specifica modalità di interazione, che può essere per esempio un paradigma di interazione basato sulla chiamata di procedure remote (RPC, Remote Procedure Call) oppure un paradigma di programmazione basata sullo scambio di messaggi.
Schematicamente possiamo collocare il middleware come mostrato nella figura a lato.



Tra le funzionalità del middleware ricordiamo:
- i servizi di astrazione e cooperazione;
- i servizi per le applicazioni;
- i servizi di amministrazione del sistema;
- il servizio di comunicazione;
- l'ambiente di sviluppo applicativo.

Un servizio di middleware è un servizio general-purpose che si colloca tra piattaforme e applicazioni (Bernstein).
Nonostante i molti benefici offerti dal middleware, questo però non è la risposta a tutti i problemi legati ai sistemi distribuiti: i problemi connessi alle decisioni "affrettate" o alla "progettazione approssimativa" e/o spesso non completamente competente degli sviluppatori. Per esempio, chi sviluppa applicazioni distribuite spesso non conosce la differenza nella realizzazione di una chiamata di procedura remota e di una chiamata locale!
A volte il middleware è focalizzato solo sulla comunicazione tra componenti e quindi non ha "nessuna responsabilità" sui problemi derivati da errori e malfunzionamenti della applicazione e non è in grado di gestire fallimenti della rete e guasti nei server.

Conclusioni

Nella seguente tabella ricapitoliamo le caratteristiche essenziali delle diverse architetture distribuite descritte sinora:

CARATTERISTICHE ESSENZIALI	ARCHITETTURE MIMD		OS NETWORK	MIDDELWARE
	Multiprocessori	Multicomputer		
Gradi di trasparenza	Molto alto	Alto	Basso	Alto
Unico SO su tutti i nodi	SÌ	SÌ	NO	NO
NR. di copie del SO	1	N	N	N
Forma di comunicazione	Memoria condivisa	Messaggi/ memoria condivisa	Files	Modelli specifici
Gestione delle risorse	Globale centralizzata	Globale distribuita	Per nodo	Per nodo
Scalabilità	NO	Modesta	SÌ	Variabile
Apertura ai diversi SO	CHIUSO	CHIUSO	APERTO	APERTO